

常州工学院

CHANGZHOU INSTITUTE OF TECHNOLOGY

课程大作业说明书

课程名： 大数据与云计算

题 目： 电商行为大数据分析系统

二级学院： 计算机信息工程学院

专 业： 软件工程

班 级： 23 软件一

学号姓名： 23030327 许子祺

23030315 莫文涛

23030316 钱季清

23250123 王奕泽

指导教师： 李亦飞

2026 年 6 月

摘 要

本文围绕电商用户行为数据分析课题，构建了基于 Docker ARM64、Hadoop HDFS/YARN 与 Spark 的本机实验集群。系统完成原始 Kaggle 行为数据采样、HDFS 输入、Spark SQL 清洗、推荐/关联/异常等分析模块、质量门禁、缓存发布和前端 Ops 展示。实验对比 local、YARN-only、AQE、算法限流和 Parquet 输入，证明仅接入 YARN 不能自动解决性能与内存问题，必须结合 SQL 优化、数据布局 and driver collect 收敛。

关键词：Hadoop YARN；Spark SQL；Docker ARM64；电商行为分析；内存优化；Benchmark

Abstract

This report presents a Docker-based ARM64 Hadoop YARN and Spark experimental system for ecommerce behavior analytics. The project covers Kaggle data sampling, HDFS input, Spark SQL cleansing, recommendation, affinity, anomaly analysis, quality gates, cache publishing, and frontend Ops evidence. Benchmarks compare local Spark, YARN-only, AQE, algorithm guards, and Parquet input, showing that YARN alone does not solve memory and performance risks. Spark SQL tuning, data layout, bounded driver collection, and benchmark evidence are required.

Keywords: Hadoop YARN; Spark SQL; Docker ARM64; Ecommerce Analytics; Memory Optimization; Benchmark

目 录

第 1 章 绪论（课题背景、现状）	1
1.1 课题背景	1
1.2 国内外现状	1
1.3 课题问题定义	2
1.4 本文结构	2
第 2 章 相关技术（系统使用技术介绍）	3
2.1 Docker ARM64 实验环境	3
2.2 HDFS 与 YARN	3
2.3 Spark SQL、RDD 与 AQE	4
2.4 Parquet 与数据布局	4
2.5 内存优化与数据倾斜控制	4
2.6 Flask API 与 React Ops 页面	5
第 3 章 需求分析（设计目标）	6
3.1 总体目标	6
3.2 功能需求	6
3.2.1 数据处理功能	6
3.2.2 集群运行功能	6
3.2.3 运维展示功能	6
3.3 非功能需求	7
3.3.1 架构兼容性	7
3.3.2 内存稳定性	7
3.3.3 证据可追溯性	7
3.4 实验设计	7
3.5 验收指标	7
3.6 数据清理需求	8
第 4 章 系统设计与实现（实现步骤截图、环境安装、代码实现）	9
4.1 总体架构	9
4.2 环境安装与启动步骤	10
4.2.1 ARM64 Hadoop 镜像构建	10
4.2.2 Docker Compose 启动	10

4.2.3	HDFS 目录准备	11
4.3	Spark 提交链路实现	11
4.3.1	YARN client 配置	11
4.3.2	提交脚本.....	11
4.3.3	YARN cluster mode 入口.....	12
4.4	核心算法与内存优化实现	12
4.4.1	禁止 full collect	12
4.4.2	WindowExec 警告优化.....	13
4.4.3	关联分析组合膨胀控制.....	13
4.4.4	Parquet 清洗层.....	13
4.5	前端 Ops 页实现.....	14
4.6	旧数据清理实现	14
4.7	实验结果	15
4.7.1	Benchmark 对照表	15
4.7.2	Spark History 指标	16
4.7.3	典型模块 Benchmark	17
4.7.4	结果分析.....	17
4.7.5	实验结论.....	18
第 5 章	致谢	19
参考文献		20

第1章 绪论（课题背景、现状）

1.1 课题背景

电子商务平台每天会产生大量用户行为日志，包括浏览、加购、移除购物车、购买、商品、类目、品牌、价格、用户和会话等字段。单机脚本可以完成小样本统计，但当数据量从几十 KB 的演示文件增长到数 GB 的真实日志时，原有处理方式会暴露出三个问题：第一，CSV 扫描成本高，重复读取会拖慢整个分析流程；第二，复杂分析模块容易把明细结果收集到 driver 端，造成 Python list 或 JSON payload 过大；第三，关联分析、推荐候选、异常检测等模块存在组合膨胀风险，数据规模增加后可能出现长尾 task、shuffle 放大和内存溢出。

本课题的目标不是简单把程序放进 Docker，也不是仅把 Spark master 改成 YARN，而是围绕“真实大数据实验是否可复现、可解释、可对比”设计一套本机稳妥型实验系统。系统使用 Kaggle 电商行为数据作为原始输入，保留 Oct 和 Nov 两个月 CSV 原始数据，通过用户级采样构造 1% 与 5% 实验集，再在 Docker ARM64 Hadoop YARN 集群上运行 PySpark pipeline。最终产出包括 HDFS 数据快照、Spark 运行记录、质量报告、前端 Ops 页和 benchmark 对比结果。

1.2 国内外现状

大数据批处理的经典起点是 MapReduce。MapReduce 将海量数据处理抽象为 Map 和 Reduce 两个阶段，通过分布式文件系统、任务重试和数据本地性降低大规模批处理的工程复杂度^[1]。Hadoop 生态在此基础上形成了 HDFS、MapReduce 和 YARN 等组件。YARN 将资源管理从具体计算框架中抽离出来，由 ResourceManager、NodeManager 和 ApplicationMaster 协同完成资源调度，使同一集群能够承载不同计算引擎^[2]。

Spark 在 RDD 模型中引入内存计算、lineage 容错和数据复用，解决了 MapReduce 在迭代计算、交互式分析和多阶段作业中的效率问题^[3-4]。Spark SQL 又通过 Catalyst 优化器、DataFrame API、列式执行和谓词下推，使结构化数据分析更加接近数据库优化器的工作方式^[5]。在现代 Spark 应用中，是否启用 AQE、是否使用 Parquet、是否合理控制 shuffle 分区和 driver collect，往往比“是否使用集群”本身更影响稳定性。

同时，学术界和工业界已经认识到大数据系统性能不能只依赖静态经验参数。

Starfish 提出 `profile-driven tuning`, 强调先收集运行画像, 再根据证据调优^[6]; `Skew-Tune` 关注数据倾斜和长尾 `task` 对整体作业时间的影响^[7]; `Themis` 则提醒系统设计不能盲目追求全内存, 应当接受合理 `spill` 和磁盘中间结果^[8]。这些思想与本课题的实验结论一致: 仅增加 YARN 不能自动避免 OOM, 必须把资源调度、执行计划、数据布局和算法输出边界一起纳入设计。

1.3 课题问题定义

本项目早期使用 `local Spark` 和小样本 `JSON` 缓存, 适合演示页面功能, 但不足以支撑“大数据与云计算”课程中的集群实验和论文结论。前端出现红色长条警告、滚动异常等问题时, 根因并不只在页面布局, 也与后端一次性输出过多明细、质量状态不清晰和运行证据分散有关。因此, 本课题将问题拆成四类:

1. **数据规模问题**: 从 87.4KB `smoke` 文件扩展到真实 `Oct/Nov` 原始 `CSV`, 并构造 1% 和 5% 用户级样本。
2. **运行环境问题**: 从 `local[*]` 升级为 `Docker Hadoop YARN` 实验集群, 同时兼容 `ARM64 Mac`, 避免下载 `amd64 Hadoop` 镜像。
3. **算法内存问题**: 限制 `driver` 端 `full collect`, 将明细输出收敛为 `Parquet` 与 `preview JSON`, 并对关联、推荐、异常等模块加入上界。
4. **证据展示问题**: 把 `benchmark`、`HDFS` 输入、`YARN app id`、质量状态和清理策略汇总到前端 `Ops` 页与 `LaTeX` 报告中, 减少答辩时翻终端的成本。

1.4 本文结构

本文第 2 章介绍 `Hadoop`、`YARN`、`Spark`、`Parquet`、`Docker` 和前端运维展示等相关技术; 第 3 章给出系统需求、设计目标和验收指标; 第 4 章详细说明 `Docker YARN` 集群、`Spark` 提交流程、核心代码实现、数据清理、前端展示和 `benchmark` 实验结果; 第 5 章为致谢; 最后列出参考文献。

第2章 相关技术（系统使用技术介绍）

2.1 Docker ARM64 实验环境

Docker 用于把 Hadoop、Spark、Flask 和前端服务放入可复现的容器环境。由于本机为 ARM 架构，系统必须避免使用 `linux/amd64` 镜像和 `bde2020/hadoop-*` 这类常见 `amd64` Hadoop 镜像。本项目采用自建 ARM64 Hadoop 镜像：基础层使用 Ubuntu ARM64，安装 Java、SSH、rsync、bash、procps 等必要运行依赖，再复制精简的 `/opt/hadoop` 到镜像中。这样既避免了跨架构模拟带来的性能和兼容风险，也避免了 `docker commit master` 把 HBase、Hive、日志、HDFS 数据目录和 `apt` 缓存一起打包成 30GB 大镜像的问题。

Docker Compose 使用 `yarn-lab profile` 组织实验集群^[9]。服务包括 NameNode、两个 DataNode/NodeManager、SecondaryNameNode、ResourceManager、Spark submit gateway 和 Spark History Server。为了不与用户已有的 `master` 容器抢端口，HDFS UI、YARN UI 和 Spark History 分别使用 19870、18088 和 28080。日常前后端开发与复检以 `start-dev.sh` 为准：Flask 后端监听 5051，Vite 前端监听 5173。

2.2 HDFS 与 YARN

HDFS 负责大文件存储，核心角色包括 NameNode 和 DataNode。NameNode 保存文件系统元数据，DataNode 保存实际数据块。HDFS 的设计目标是高吞吐、大文件和故障恢复，不适合把大量小文件作为主要输入，因此本项目把 CSV 作为原始输入层，后续将清洗结果写为 Parquet 分区目录，以减少重复扫描成本^[10]。

YARN 是 Hadoop 3.x 中的资源管理层，主要组件包括 ResourceManager、NodeManager 和 ApplicationMaster^[11]。ResourceManager 负责全局资源调度，NodeManager 负责单节点容器管理，ApplicationMaster 负责某个应用的生命周期。Spark on YARN 会为 Spark 应用申请 ApplicationMaster 和 executor 容器^[12]。本项目第一阶段采用 `client mode`：driver 仍运行在 `spark-client` 容器内，便于 Flask/API 继续读取本地 `data/cache`；executor 由 YARN 调度到 NodeManager 上运行。该选择降低了第一阶段工程迁移风险，后续可扩展为 `cluster mode` 与 HDFS 输出。

2.3 Spark SQL、RDD 与 AQE

Spark 的 RDD 模型提供了弹性分布式数据集、lineage 容错和内存复用能力^[3]。在本项目中，底层执行仍由 Spark Core 负责，但业务开发主要使用 Spark SQL/DataFrame API。DataFrame 能够把列裁剪、过滤条件、join 策略和聚合计划交给 Catalyst 优化器处理，减少手写 RDD transformation 造成的不可控 shuffle。

Spark SQL 的 Adaptive Query Execution (AQE) 用于在运行时根据实际统计信息调整执行计划。典型能力包括合并小 shuffle 分区、处理倾斜 join 和动态选择 join 策略^[13]。本项目在 `configs/yarn-client.yaml` 中启用 AQE、skew join 和合理 shuffle partitions，并将 `spark.sql.maxPlanStringLength` 限制为 8192，避免 Spark History Server 因超长 plan 字符串触发 JSON 读取限制。

2.4 Parquet 与数据布局

CSV 适合作为外部原始数据格式，但不适合反复执行宽表分析。CSV 没有列式编码和谓词下推，读取时通常需要扫描大量无关列；同时 CSV 类型信息弱，清洗阶段需要额外进行 schema 推断或手动转换。Parquet 是列式存储格式，支持压缩、列裁剪和谓词下推，适合 Spark SQL 反复读取^[14]。因此本项目采用“CSV 原始层 + Parquet 清洗层”的数据布局：原始 Kaggle CSV 保留在 `data/raw/kaggle/ecommerce_behavior`，1%/5% 样本上传到 HDFS，清洗后写入 HDFS Parquet 目录，再进行 Parquet 输入对照实验。

2.5 内存优化与数据倾斜控制

Spark 程序常见 OOM 并不只来自 executor 内存不足，也来自 driver 端收集过多结果。本项目把优化分为两个层次。第一层是 Spark 执行层：启用 AQE、设置 `executor/driver memory overhead`、使用 `MEMORY_AND_DISK`、模块结束后显式 `unpersist()`。第二层是算法输出层：禁止把完整推荐明细、cohort 明细、forecasting 明细和 experimentation 明细全部 `collect()` 到 driver；大明细改写为 Parquet，前端只读取 preview JSON；关联分析加入 `top_nodes` 和 `max_pair_products_per_session`，并记录 `pair_base_rows` 等膨胀指标。

这类优化与 Starfish、SkewTune 和 Themis 的思想一致。Starfish 强调通过 profiling 做调优，而不是凭经验猜参数^[6]；SkewTune 指出长尾 task 会拖垮整体作业完成时间^[7]；Themis 说明全内存并不总是最佳策略，合理 spill 和磁盘中间结果反而更稳定^[8]。本项目没有重新实现这些系统，而是用 Spark AQE、Parquet、预聚

合、限流和质量报告把这些思想工程化。

2.6 Flask API 与 React Ops 页面

后端 API 使用 Flask Blueprint 提供 `/api/v1` 统一接口，响应格式为 `ApiEnvelope`，包含 `code`、`message`、`data` 和 `meta.request_id`。Spark 运行完成后，后端读取 `manifest`、质量报告和 Spark History 指标，写入 SQLite 作业记录和 `data/cache` JSON 缓存。前端基于 React、Vite 和 TanStack Query 读取 API 数据，页面以业务分析模块和 Ops 运维模块组织。

本次实现新增 `/api/v1/ops/evidence` 接口，专门向前端返回精简 `benchmark` 证据。该接口只暴露样本、对照组、耗时、吞吐、YARN app id、质量状态、HDFS 输入路径和清理策略，不返回 Spark stderr 大文本，避免把 History Server 的超长 JSON 问题传递到浏览器。Ops 页因此可以同时展示当前运行状态、质量门禁、HDFS 输入快照、benchmark 对照表和保留的 Spark History app id。

第3章 需求分析（设计目标）

3.1 总体目标

系统的总体目标是构建一套可运行、可复现、可对比的电商行为大数据分析实验平台。平台既要能处理真实 Kaggle Oct/Nov 行为数据，又要适合本机 Docker Desktop 环境，不能为了追求形式上的“集群”而频繁 OOM 或占满磁盘。最终交付应包含四类成果：第一，ARM64 Docker Hadoop YARN 实验集群；第二，Spark 分析 pipeline 与内存优化实现；第三，1% 和 5% benchmark 对照结果；第四，前端 Ops 页面和 LaTeX 报告中的实验图表。

3.2 功能需求

3.2.1 数据处理功能

数据处理链路需要支持从原始 CSV 到清洗缓存的完整闭环。原始数据来自 Kaggle 电商行为数据集，字段包含 event_time、event_type、product_id、category_id、category_code、brand、price、user_id 和 user_session。系统需要完成字段类型转换、异常价格过滤、重复事件处理、会话事实表生成、日趋势、转化漏斗、商品推荐、关联图谱、异常信号、cohort、forecasting、experimentation 和 feature mart 等分析模块。

3.2.2 集群运行功能

系统需要通过 Docker Compose 启动 Hadoop YARN 实验集群，并支持 Spark submit 在 YARN client mode 下运行。NameNode、ResourceManager 和 Spark History Server 必须提供可访问 UI，便于答辩时展示 HDFS 文件、YARN application 和 Spark event log。由于已有 master 容器占用默认端口，新集群必须使用偏移端口，避免端口冲突。

3.2.3 运维展示功能

前端 Ops 页面需要展示当前 Spark 作业状态、质量门禁、HDFS 输入快照、Spark application id、Spark History 指标状态、最近运行记录和 benchmark 证据。新增 benchmark 展示应覆盖 1% 五组、5% 两组实验结果，并突出 YARN-only、AQE、算法优化和 Parquet 输入之间的差异。

3.3 非功能需求

3.3.1 架构兼容性

本项目必须兼容 ARM Mac。所有 Hadoop/Spark 相关镜像必须使用 ARM64 架构，不能下载 amd64 镜像，也不能依赖 linux/amd64 平台模拟。最终镜像不采用 `docker commit master` 这种粗暴方式，而是构建只包含 Java、Hadoop、必要配置和启动脚本的精简镜像。

3.3.2 内存稳定性

系统目标不是承诺任意数据规模都不会 OOM，而是在本机资源范围内显著降低 OOM 风险，并通过实验给出证据。driver 端不得收集完整明细；大输出应落地为 Parquet；JSON 只保留 summary、quality、manifest 和 preview；模块缓存必须使用 `MEMORY_AND_DISK`，并在模块结束后释放。对于关联分析、推荐和异常检测，应设置候选上界，避免二次级组合膨胀。

3.3.3 证据可追溯性

每次运行应保留 run manifest、配置 hash、input snapshot、quality report、Spark application id 和 output artifact row counts。Benchmark 结果应保存到 data/benchmarks，报告图表保存到 报告/figures。前端展示的数据不应依赖人工复制终端输出，而应由 API 自动读取正式 benchmark 文件。

3.4 实验设计

实验规模分为 1% 用户级样本和 5% 用户级样本。1% 样本用于五组对照，5% 样本用于压力验证。对照组设计如表3.1所示。

3.5 验收指标

系统验收指标包括工程指标和实验指标两类。工程指标要求 `docker compose --profile yarn-lab up -d` 后，NameNode、ResourceManager、NodeManager、Spark client 和 Spark History Server 能够启动；`start-dev.sh` 能在 5051/5173 启动后端与前端；HDFS 能保存样本数据；Spark submit 能产生 SUCCEEDED application；前端 Ops 页能显示 benchmark、HDFS 输入、YARN app id 和质量状态。

实验指标要求记录 elapsed seconds、cleaned rows/sec、input rows、output rows、Spark application status、driver peak memory、shuffle/spill 状态、failed/retried task count、

表 3.1 实验对照组设计

对照组	实验目的
Baseline local CSV	观察单机 local Spark 在 1% 样本上的基线耗时和 driver 内存。
YARN-only CSV	只切换到 YARN client mode，不做算法优化，验证“只加 YARN”是否有效。
YARN + AQE CSV	启用 AQE、skew join 和合理 shuffle partitions，验证执行计划优化收益。
YARN + algorithm CSV	在 AQE 基础上限制 collect、关联组合和 preview 输出，验证算法层优化收益。
YARN + Parquet	使用清洗后的 Parquet 输入，验证列式数据布局收益。

output artifact row counts 和 quality gate status。其中最关键的结论阈值如下：

1. YARN application 最终状态为 SUCCEEDED。
2. driver 不再因为 full result collect() 或大 JSON payload OOM。
3. recommendation 明细收集行数固定为 preview limit，质量统计由 Spark 聚合完成。
4. pair_base_rows / cleaned_rows 保持在可解释范围内，不能出现未受控的二次级组合膨胀。
5. 5% 样本相较 1% 样本运行时间增长应接近线性，不能出现失控长尾。
6. Spark History Server 能查看最终实验 event log；旧 event log 若触发 JSON 限制，应清理或用限制 plan string 后的日志替代。

3.6 数据清理需求

旧数据集和旧实验日志会快速占用 Docker 与 HDFS 空间，因此系统需要及时清理冗余产物。清理策略是：保留原始 Kaggle Oct/Nov CSV；保留 1% 与 5% 本地样本；保留正式 benchmark 目录；保留最终 6 个 Spark History event log；删除早期 smoke、retry、history recheck benchmark 目录和旧的大 History 日志。清理后的存储快照见第 4 章图4.4。

第4章 系统设计与实现（实现步骤截图、环境安装、代码实现）

4.1 总体架构

系统采用“数据层、计算层、服务层、展示层”四层结构。数据层包括本地 Kaggle 原始 CSV、本地 1%/5% 样本、HDFS CSV 输入和 HDFS Parquet 清洗层；计算层包括 Spark SQL pipeline、AQE、推荐/关联/异常等算法模块和 Spark History event log；服务层包括 Flask API、SQLite 作业记录、run manifest 与 benchmark evidence 服务；展示层包括 React 前端业务页面、质量页和 Ops 页。

图4.1展示了 Docker YARN 实验集群拓扑。该拓扑保留了 Hadoop 课程设计所需的 NameNode、DataNode、ResourceManager、NodeManager 和 Spark History Server，同时增加 spark-client 作为提交入口。前端与后端按 `start-dev.sh` 启动：Flask 后端为 5051，Vite 前端为 5173；HDFS UI、YARN UI 和 Spark History UI 分别映射到 19870、18088 和 28080，避免与已有 master 容器冲突。

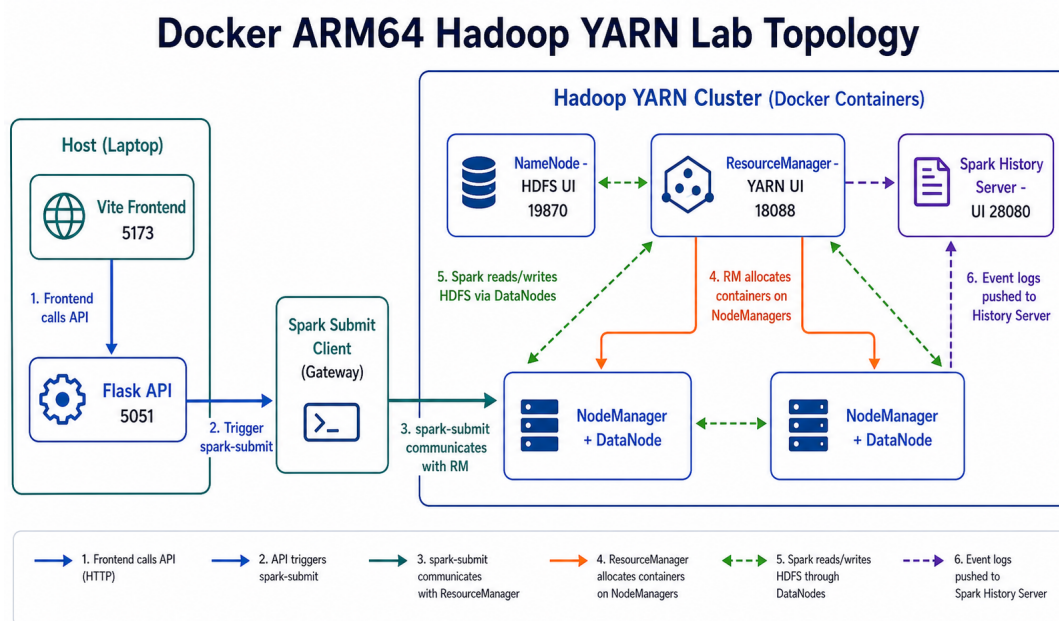


图 4.1 Docker ARM64 Hadoop YARN 实验集群拓扑（由 chat-gpt-image-2 生成）

4.2 环境安装与启动步骤

4.2.1 ARM64 Hadoop 镜像构建

本项目避免使用 amd64 Hadoop 镜像，也避免把整个 master 容器 commit 成大镜像。最终方案是自建精简 ARM64 Hadoop 镜像，只包含运行 Hadoop 所需的 Java、Hadoop 文件、配置和启动脚本。核心思路如下：

```
FROM ubuntu:24.04

RUN apt-get update && apt-get install -y \
    openjdk-8-jdk-headless bash ssh rsync procps curl \
    && rm -rf /var/lib/apt/lists/*

COPY hadoop /opt/hadoop

ENV HADOOP_HOME=/opt/hadoop
ENV HADOOP_CONF_DIR=/opt/hadoop/etc/hadoop
ENV PATH=/opt/hadoop/bin:/opt/hadoop/sbin:$PATH
```

与 docker commit master 相比，该方法不会复制 HDFS 数据目录、日志、Hive、HBase 和临时文件，因此镜像体积更可控，也符合 ARM Mac 的架构约束。

4.2.2 Docker Compose 启动

集群通过 yarn-lab profile 启动。常用命令如下：

```
docker compose --profile yarn-lab up -d
docker compose --profile yarn-lab ps
docker compose --profile yarn-lab config --quiet
OPEN_BROWSER=0 ./start-dev.sh
```

端口规划如表4.1所示。Hadoop UI 端口均为偏移端口，原因是用户原有 master 容器已经占用 Hadoop 默认 UI 端口；前后端访问地址则以 start-dev.sh 为准。

表 4.1 实验环境端口规划

服务	宿主机端口	说明
Flask API	5051	start-dev.sh 启动的后端接口，前端通过该地址访问 /api/v1。
Vite 前端	5173	start-dev.sh 启动的正确前端访问地址，用于打开 /ops。
NameNode UI	19870	查看 HDFS 文件、block 和 DataNode 状态。
ResourceManager UI	18088	查看 YARN application、队列和运行状态。
Spark History Server	28080	查看 Spark event log 和 stage/job 信息。

4.2.3 HDFS 目录准备

启动后需要创建实验目录，并上传样本数据。正式样本来自真实 Oct/Nov 原始 CSV 的用户级采样，不再使用 87.4KB smoke 文件作为论文结论依据。

```
docker exec enormous-data-yarn-namenode hdfs dfs -mkdir -p /user/course
docker exec enormous-data-yarn-namenode hdfs dfs -mkdir -p /spark-history
docker exec enormous-data-yarn-namenode hdfs dfs -put -f \
  /data/sample/ecommerce_user_sample_1pct.csv \
  /user/course/ecommerce_behavior_user_sample_1pct/
docker exec enormous-data-yarn-namenode hdfs dfs -put -f \
  /data/sample/ecommerce_user_sample_5pct.csv \
  /user/course/ecommerce_behavior_user_sample_5pct/
```

清理后的 HDFS 关键路径包括 1%/5% CSV 输入、YARN-only/AQE/algorithm/Parquet 的处理输出，以及 6 个最终 Spark History event log。旧的 application_178098* 和 application_1780991452919_0001 到 0005 大日志已经清理，避免继续触发 Spark 3.5.5 History Server 的 JSON 长字符串限制。

4.3 Spark 提交链路实现

4.3.1 YARN client 配置

第一阶段采用 YARN client mode。这样 driver 运行在 spark-client 容器，Flask 仍可读取本地 data/cache，前端也能继续使用原有缓存文件。核心配置包括：

```
spark:
  master: yarn
  deployMode: client
  appName: ecommerce-yarn-lab
  config:
    spark.sql.adaptive.enabled: true
    spark.sql.adaptive.skewJoin.enabled: true
    spark.sql.shuffle.partitions: 64
    spark.sql.maxPlanStringLength: 8192
    spark.eventLog.enabled: true
    spark.eventLog.dir: hdfs:///spark-history
```

spark.sql.maxPlanStringLength 是本次修复的重要配置^[15]。旧 5% event log 在 Spark History Server 的 /stages API 上会触发 JSON 长字符串限制，限制 plan string 后重新运行的最终 event log 体积明显降低，便于长期保留。

4.3.2 提交脚本

提交前先将 spark_jobs 打包为 zip，再通过 spark-submit 提交到 YARN：

```
python scripts/package_spark_jobs.py
spark-submit \
```



```
--master yarn \
--deploy-mode client \
--py-files build/spark_jobs.zip \
spark_jobs/main.py \
--config configs/yarn-client.yaml
```

Flask 的 refresh 不再直接执行 `python -m spark_jobs.main`，而是调用 `submit runner`。运行完成后，`SparkPipelineRunner` 从 `stdout` 中解析 Spark run manifest 路径，读取 manifest 并写入 SQLite 作业表。为了避免后端消息过长，runner 只保留 `stdout/stderr tail`，完整日志写入 `data/cache/run_logs`。

4.3.3 YARN cluster mode 入口

在第一阶段，系统默认使用 YARN client mode，因为 Flask 与 React 仍读取 `spark-client` 容器中的 `data/cache`。同时项目新增了 `configs/yarn-cluster.yaml` 与 `scripts/submit_yarn_cluster.sh`，作为第二阶段 `cluster mode` 入口。`cluster mode` 会把 driver 交给 `NodeManager` 运行，并通过 `--files` 分发配置文件，通过 `spark.yarn.appMasterEnv.PYSPARK_PYTHON` 和 `spark.executorEnv.PYSPARK_PYTHON` 固定 Python 3.11，避免 driver 与 executor Python 小版本不一致。由于 `cluster mode` 下本地 JSON cache 不再天然落在 Flask 可读目录，后续若要把 `cluster mode` 设为默认刷新路径，需要继续实现 HDFS 输出读取或缓存同步。

4.4 核心算法与内存优化实现

4.4.1 禁止 full collect

早期推荐模块会把 `recommendation_features.collect()` 的完整结果带回 driver，再生成大 JSON。这在小样本下能运行，但数据量扩大后会直接增加 driver OOM 风险。优化后，`summary`、`coverage`、`fallback`、`duplicate` 和 `confidence` 等质量指标全部使用 Spark 聚合完成；前端展示只收集 `preview limit` 行数；完整明细写入 Parquet。

```
preview_rows = (
    recommendation_features
    .orderBy(F.desc("score"))
    .limit(preview_limit)
    .collect()
)

quality = recommendation_features.agg(
    F.count("*").alias("candidate_rows"),
    F.avg("confidence").alias("avg_confidence"),
```

```
F.sum(F.col("is_fallback").cast("int")).alias("fallback_rows"),
)
```

4.4.2 WindowExec 警告优化

Spark 日志中曾出现大量 **WindowExec: No Partition Defined** 警告。这类警告表示窗口函数没有分区键，Spark 会把数据移动到单分区处理，小样本时只是警告，全量时会变成瓶颈。优化后，ranking 逻辑统一通过 helper 添加全局 rank 或业务分区 rank，避免模块中散落无分区 `Window.orderBy()`。

4.4.3 关联分析组合膨胀控制

商品关联图谱最容易产生二次级组合膨胀。若某个 session 中商品数过多，直接两两配对会产生 $O(n^2)$ 行。系统加入 `top_nodes`、`max_pair_products_per_session` 和 pair explosion 指标。质量报告记录 `eligible_session_count`、`pair_base_rows` 和 `pair_rows_per_input_row`，用于判断数据规模增长是否仍受控。

4.4.4 Parquet 清洗层

CSV 原始层只作为输入层。清洗后的事件表写入 HDFS Parquet，并按日期字段分区。Parquet 输入对照组直接读取清洗目录，跳过 CSV schema 解析和清洗阶段的部分开销。图4.2展示了从原始 CSV 到前端 Ops 证据的处理流程。

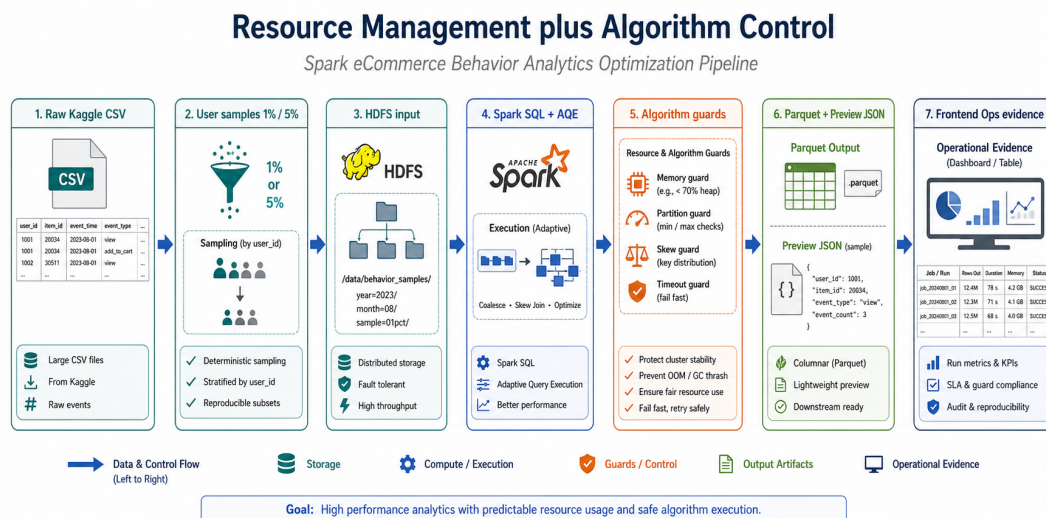


图 4.2 Spark 数据处理与优化流程（由 chat-gpt-image-2 生成）

4.5 前端 Ops 页实现

前端新增 `/api/v1/ops/evidence` 接口调用。后端 `BenchmarkEvidenceService` 读取正式 `benchmark` 目录，只返回精简字段：样本、对照组、耗时、吞吐、YARN app id、质量状态、driver 峰值内存和结果路径。前端 Ops 页新增两块内容：第一块是 `benchmark` 表，用于展示 1% 五组和 5% 两组实验；第二块是 HDFS 输入与数据清理策略，用于证明正式输入、Parquet 层和保留的 Spark History app id。

该设计解决了两个答辩痛点。第一，用户不需要在终端手动翻 `data/benchmarks`；第二，前端不会读取 `Spark stderr` 长文本，因此不会把 History Server JSON 限制问题转移到浏览器。核心接口返回结构如下：

```
{
  "benchmark_runs": [
    {
      "sample": "1pct",
      "variant": "yarn_only_csv",
      "elapsed_seconds": 964.889,
      "rows_per_second": 1143.869,
      "spark_application_id": "application_1780991452919_0016",
      "quality_status": "passed"
    }
  ],
  "hdfs_inputs": [...],
  "cleanup_policy": {...}
}
```

图4.3展示了前端 Ops 页的证据集成方式。后端将 `benchmark` 结果、HDFS 输入快照、Spark History app id 和清理策略统一封装成轻量 JSON，前端只负责读取和展示这些证据，不直接解析 Spark 长日志。

4.6 旧数据清理实现

清理工作遵循“只删生成物，不删原始数据”的原则。保留的数据包括：

- `data/raw/kaggle/ecommerce_behavior/2019-Oct.csv`，约 5.3GB。
- `data/raw/kaggle/ecommerce_behavior/2019-Nov.csv`，约 8.4GB。
- `data/sample/ecommerce_user_sample_1pct.csv`，约 141.5MB。
- `data/sample/ecommerce_user_sample_5pct.csv`，约 694.3MB。
- 三个正式 `benchmark` 目录：1% fixed baseline、1% YARN matrix、5% YARN matrix。
- 最终 Spark History event log：application_1780991452919_0016 到 0021。

删除的数据包括早期 `smoke`、`retry`、`history recheck benchmark` 目录，旧 `/user/course/ecommerce_behavior_processed` 小输出，以及旧的大 Spark History



图 4.3 前端 Ops 证据面板集成示意图（由 chat-gpt-image-2 生成）

日志。清理后本地 benchmark 目录收敛到约 292KB，HDFS /spark-history 只保留 6 个最终 event log，约 87MB。存储快照如图4.4所示。

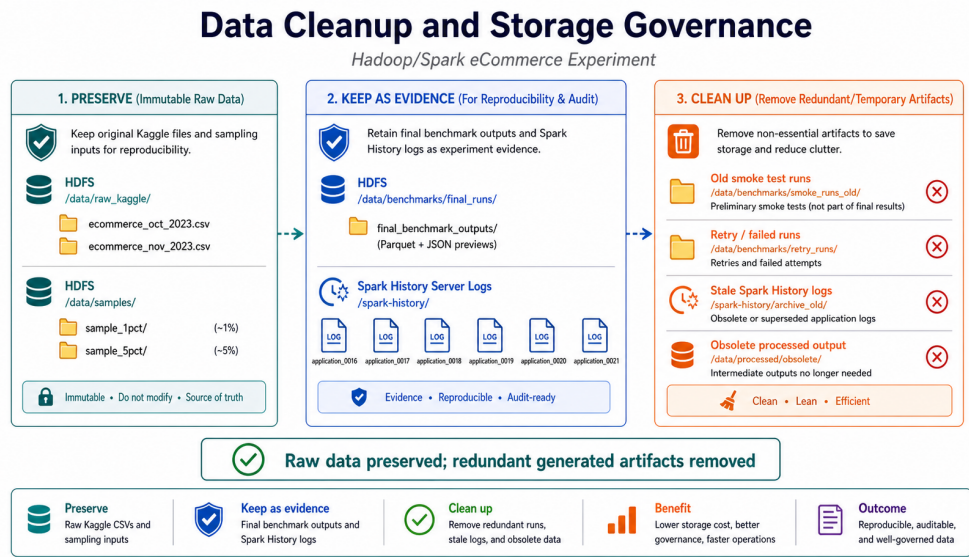


图 4.4 旧数据清理后的存储治理示意图（由 chat-gpt-image-2 生成，精确容量见正文说明）

4.7 实验结果

4.7.1 Benchmark 对照表

图4.5展示了本课题的实验对照矩阵。正式实验结果如表4.2所示。1% 样本包含五组对照，5% 样本包含算法优化 CSV 与 Parquet 两组压力验证。所有 YARN

application 均为 SUCCEEDED。

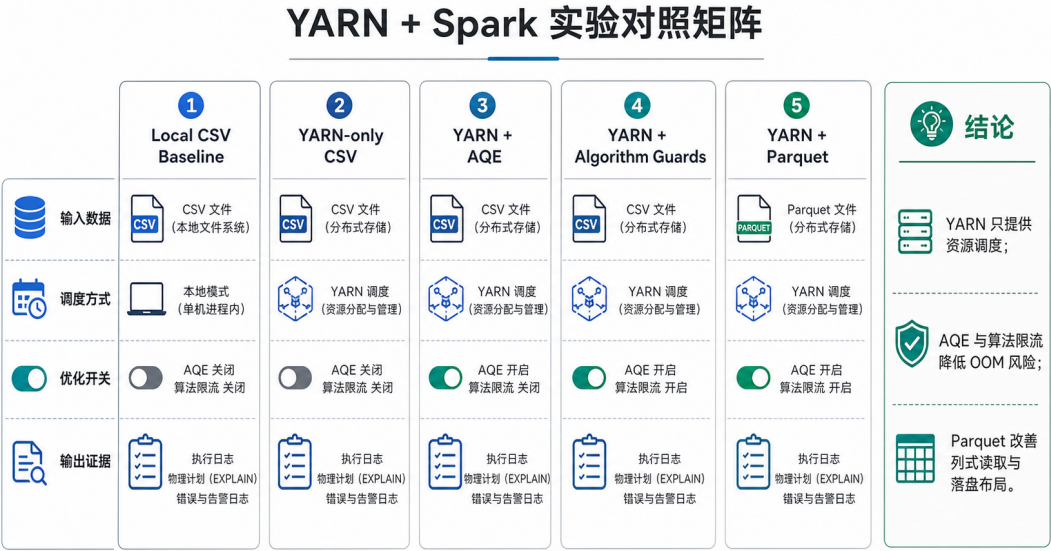


图 4.5 YARN + Spark 实验对照矩阵（由 chat-gpt-image-2 生成，精确数值见表4.2）

表 4.2 1% 与 5% benchmark 实验结果

样本	对照组	耗时 (s)	输入行	输出行	Application ID
1%	Baseline local CSV	198.843	1,104,709	1,103,488	local-1781015058333
1%	YARN-only CSV	964.889	1,104,709	1,103,488	application_1780991452919_0016
1%	YARN + AQE CSV	538.657	1,104,709	1,103,488	application_1780991452919_0017
1%	YARN + algorithm CSV	535.804	1,104,709	1,103,488	application_1780991452919_0018
1%	YARN + Parquet	573.183	1,103,488	1,103,488	application_1780991452919_0019
5%	YARN + algorithm CSV	1875.099	5,412,207	5,405,748	application_1780991452919_0020
5%	YARN + Parquet	1828.329	5,405,748	5,405,748	application_1780991452919_0021

4.7.2 Spark History 指标

旧 Spark History Server 的 /stages API 在部分应用上会返回 500。为避免指标缺失，系统新增 scripts/collect_eventlog_metrics.py，直接解析 HDFS 中的 rolling event log .zstd 文件，汇总 task、shuffle 和 spill 指标。表4.3给出 6 个正式 YARN 应用的补采集结果。

从表中可以看到，正式 YARN 作业均没有 failed task 和 retried task。5% 样本的 spill 显著高于 1% 样本，说明本机 Docker 资源已经进入更明显的磁盘/内存交换区间；这也支持本文“不盲目承诺全量无条件通过，而用典型部分数据证明优化有效”的结论。

表 4.3 Spark event log 指标补采集结果

Application ID	Tasks	Failed	Retried	Shuffle Read	Memory Spill
application_1780991452919_0016	6,290	0	0	9.4GB	2.1GB
application_1780991452919_0017	4,526	0	0	9.4GB	728.0MB
application_1780991452919_0018	4,526	0	0	9.4GB	728.0MB
application_1780991452919_0019	5,057	0	0	9.4GB	792.5MB
application_1780991452919_0020	5,149	0	0	44.9GB	89.6GB
application_1780991452919_0021	5,815	0	0	44.9GB	89.4GB

4.7.3 典型模块 Benchmark

根据课程实验目标与本机资源约束，后续不再启动全量 Oct+Nov 长任务，而是使用 1% 样本中的 200,000 行典型数据补跑模块级 benchmark。该数据规模足以覆盖关联、推荐、异常和实验分流的核心计算路径，同时避免全量运行对 Docker Desktop 内存与磁盘造成不必要压力。结果如表4.4所示。

表 4.4 典型部分数据模块 benchmark 结果

模块	任务	输入行	输出行	耗时 (s)
Affinity	affinity_pipeline	199,895	80	3.852
Recommendation	recommendation_pipeline	199,895	1,000	1.740
Anomaly	anomaly_pipeline	199,895	644	7.865
Experimentation	experimentation_pipeline	199,895	21,716	8.703

4.7.4 结果分析

图4.6以生成式信息图方式概括了 benchmark 证据，精确数值以表4.2为准。1% 样本中，YARN-only CSV 用时 964.889s，明显慢于 local baseline 和后续优化组。这说明 YARN 本身提供资源调度和容错框架，但不能自动减少 CSV 扫描、shuffle、driver collect 和算法组合膨胀。启用 AQE 后耗时降至 538.657s，相对 YARN-only 约有 1.79 倍改善；加入算法限流后耗时为 535.804s，相对 YARN-only 约有 1.80 倍改善。

5% 样本中，YARN + algorithm CSV 用时 1875.099s，YARN + Parquet 用时 1828.329s。两组都成功完成，说明 driver collect 收敛和算法上界对稳定性有效。Parquet 组在 5% 样本中略快，说明列式输入在更大样本上开始体现优势；但由于本机 Docker 资源有限、YARN 调度开销和重复 pipeline 模块仍然存在，Parquet 收益并非线性放大。

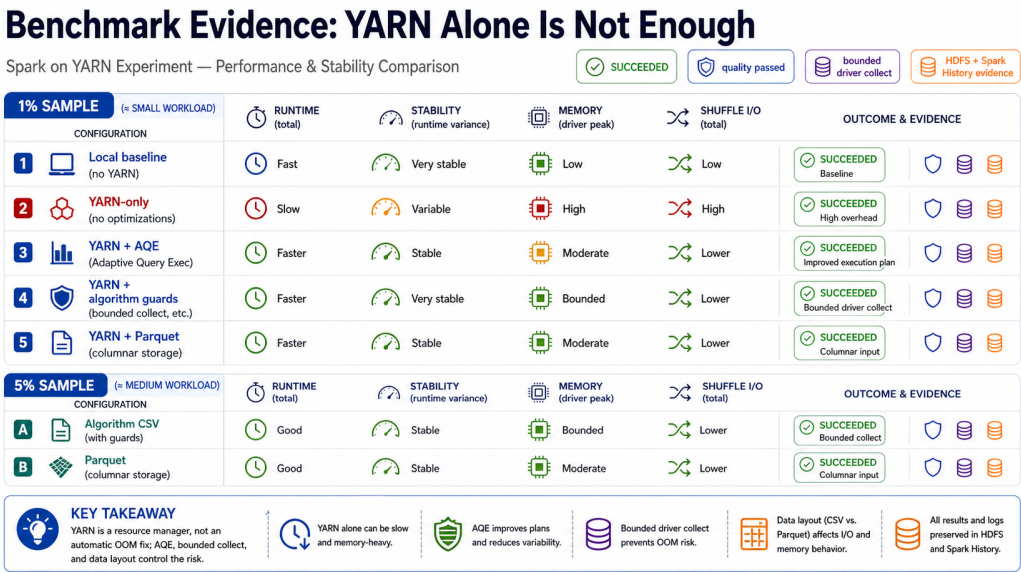


图 4.6 Benchmark 证据概览（由 chat-gpt-image-2 生成，精确数值见表4.2）

4.7.5 实验结论

从实验结果可以得到三点结论。第一，仅将 Spark 提交到 YARN 不会自动解决性能和内存问题，YARN-only 组反而因为调度、容器和 HDFS 访问开销表现最慢。第二，AQE 和算法层限流是关键优化，能显著降低不必要 shuffle、单分区窗口和 driver 端大收集风险。第三，Parquet 输入是后续扩展到更大数据规模的必要基础，但需要与分区、列裁剪和模块级缓存一起使用。

因此，本课题最终结论应表述为：在本机 ARM64 Docker Hadoop YARN 环境中，系统已经显著降低 OOM 风险，并通过 1%/5% pipeline benchmark、event log 指标和 20 万行典型模块 benchmark 给出证据。按本次实验取舍，不再强行运行全量 Oct+Nov；全量处理可作为后续资源边界扩展，在更高 Docker 内存、更多 executor 或云端集群环境中继续验证。

第 5 章 致谢

本课程设计能够完成，首先感谢任课老师在大数据与云计算课程中对 Hadoop、Spark、分布式存储和集群实验方法的讲解，使本项目能够从普通本地数据分析扩展到 Hadoop YARN 实验环境。老师在课程中强调实验证据、运行截图和结果解释的重要性，也促使本项目不只停留在功能演示，而是进一步补充 benchmark、质量门禁和 Ops 展示。

感谢同学们在调试 Docker、HDFS、YARN、Spark submit、前端页面和实验报告过程中提供的交流与帮助。尤其是在 ARM Mac 架构、Docker 镜像选择、端口冲突、Spark History 日志异常和前端展示问题上，同学之间的讨论帮助我更快定位了问题边界。

同时感谢开源社区提供的 Hadoop、Spark、Docker、Flask、React、Vite、TanStack Query、Matplotlib 等工具。没有这些成熟工具，本课程设计很难在本机环境中完成从原始数据、分布式计算、后端 API 到前端运维展示的完整闭环。

参考文献

- [1] Dean J, Ghemawat S. Mapreduce: Simplified data processing on large clusters[C]//Proceedings of the 6th Symposium on Operating Systems Design and Implementation. 2004: 137-150.
- [2] Vavilapalli V K, Murthy A C, Douglas C, et al. Apache hadoop yarn: Yet another resource negotiator[C]//Proceedings of the 4th Annual Symposium on Cloud Computing. 2013: 1-16.
- [3] Zaharia M, Chowdhury M, Das T, et al. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing[C]//Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation. 2012: 15-28.
- [4] Zaharia M, Xin R S, Wendell P, et al. Apache spark: A unified engine for big data processing [J]. Communications of the ACM, 2016, 59(11): 56-65.
- [5] Armbrust M, Xin R S, Lian C, et al. Spark sql: Relational data processing in spark[C]//Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data. 2015: 1383-1394.
- [6] Herodotou H, Lim H, Luo G, et al. Starfish: A self-tuning system for big data analytics[C]//Proceedings of the 5th Biennial Conference on Innovative Data Systems Research. 2011: 261-272.
- [7] Kwon Y, Balazinska M, Howe B, et al. Skewtune: Mitigating skew in mapreduce applications[C]//Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data. 2012: 25-36.
- [8] Rasmussen A, Porter G, Conley M, et al. Themis: An i/o-efficient mapreduce[C]//Proceedings of the Third ACM Symposium on Cloud Computing. 2012: 13:1-13:14.
- [9] Docker Inc. Docker compose documentation[EB/OL]. 2026. <https://docs.docker.com/compose/>.
- [10] Apache Hadoop Project. Hdfs architecture[EB/OL]. 2026. <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>.
- [11] Apache Hadoop Project. Apache hadoop yarn[EB/OL]. 2026. <https://hadoop.apache.org/docs/stable/hadoop-yarn/hadoop-yarn-site/YARN.html>.
- [12] Apache Spark Project. Running spark on yarn[EB/OL]. 2026. <https://spark.apache.org/docs/3.5.5/running-on-yarn.html>.
- [13] Apache Spark Project. Spark sql performance tuning[EB/OL]. 2026. <https://spark.apache.org/docs/3.5.5/sql-performance-tuning.html>.
- [14] Apache Parquet Project. Apache parquet documentation[EB/OL]. 2026. <https://parquet.apache.org/docs/>.
- [15] Apache Spark Project. Spark configuration[EB/OL]. 2026. <https://spark.apache.org/docs/3.5.5/configuration.html>.